Nama: Jonathan Teguh Samuel Kaeng

PROGRESS I
KONTROL MOTOR DASAR

1.1 Tujuan Modul

Tahap Progress 1 bertujuan untuk menguji fungsionalitas dasar perangkat keras, yaitu *driver* motor (L298N) dan mikrokontroler ESP32, secara langsung. Sistem ini beroperasi secara lokal (*wired*) menggunakan perintah teks melalui Serial Monitor di Arduino IDE tanpa melibatkan konektivitas internet (Wi-Fi) maupun *platform* IoT. Pengujian ini sangat krusial sebagai tahap kalibrasi awal guna memastikan tidak ada cacat komponen fisik maupun kesalahan pengkabelan (*wiring*).

1.2 Skema Rangkaian

Sistem menggunakan pin GPIO (*General Purpose Input/Output*) pada ESP32 yang dihubungkan ke modul *driver* motor L298N dengan pemetaan kabel sebagai berikut:

- Pin 27 (motor1Pin1): Dihubungkan ke pin IN1 pada *driver* motor (berfungsi mengatur arah putaran motor).
- Pin 26 (motor1Pin2): Dihubungkan ke pin IN2 pada *driver* motor (berfungsi mengatur arah putaran motor).

- Pin 12 (enable1Pin): Dihubungkan ke pin ENA pada *driver* motor untuk menerima sinyal PWM (*Pulse Width Modulation*).

1.3 Konsep & Logika Sistem

Untuk mengatur kecepatan putaran motor, mikrokontroler tidak sekadar memberikan tegangan statis ON/OFF (HIGH/LOW), melainkan menggunakan sinyal PWM. Pada program ini, PWM dikonfigurasi dengan frekuensi 30.000 Hz dan resolusi 8-bit.

Resolusi 8-bit berarti mikrokontroler membagi kecepatan putaran motor ke dalam rentang nilai dari 0 hingga 255, di mana:

- Nilai 0 = Motor berhenti total (0% *duty cycle*).
- Nilai 255 = Motor berputar pada kecepatan maksimal (100% *duty cycle*).

1.4 Daftar Perintah

Mikrokontroler diprogram untuk membaca input teks dari pengguna melalui Serial Monitor (dengan *baudrate* 115200) guna mengeksekusi kendali motor. Daftar perintah yang dikenali oleh sistem adalah:

- 1 : Motor ON. Menyalakan motor. Jika kecepatan sebelumnya adalah 0, sistem otomatis memberikan nilai PWM awal sebesar 150 agar motor memiliki daya angkat mula (*starting torque*).
- 0 : Motor OFF. Menghentikan putaran motor secara instan (set PWM ke 0).
- + : Naikkan Kecepatan. Menambah nilai kecepatan PWM sebesar 10 poin secara bertahap hingga mencapai batas maksimal (255).
- - : Turunkan Kecepatan. Mengurangi nilai kecepatan PWM sebesar 10 poin secara bertahap hingga mencapai batas minimal (0).
- PWM:[nilai] : Set Kecepatan Manual. Menetapkan nilai kecepatan secara spesifik. (Contoh: Mengetik PWM:200 akan langsung memutar motor pada kecepatan 200).

1.5 Source Code dan Penjelasan Logika Program

```
// PROGRESS 1 : KONTROL MOTOR DASAR / Menggunakan Kabel
// Fitur: ON/OFF, Naik/Turun PWM via Serial Monitor

// --- KONFIGURASI PIN ---
int ledPin = 2;         // Indikator LED internal ESP32
int motor1Pin1 = 27;    // Pin IN1 Driver Motor
int motor1Pin2 = 26;    // Pin IN2 Driver Motor
int enable1Pin = 12;    // Pin ENA (PWM) Driver Motor

// --- PENGATURAN PWM ---
const int freq = 30000;      // Frekuensi PWM
const int resolution = 8;    // Resolusi 8-bit (0-255)
int dutyCycle = 200;         // Nilai kecepatan awal
const int DUTY_STEP = 10;    // Jarak kenaikan/penurunan kecepatan
const int DUTY_MIN  = 0;
const int DUTY_MAX  = 255;

bool isMotorRunning = false; // Status motor (menyala/mati)
String inputString = "";     // Penampung teks dari Serial Monitor

//Fungsi Kontrol Motor

// Menerapkan nilai PWM ke hardware jika motor dalam status menyala
void applyMotor() {
  if (isMotorRunning) {
    digitalWrite(motor1Pin1, HIGH);
    digitalWrite(motor1Pin2, LOW);
    ledcWrite(enable1Pin, dutyCycle);
  }
}

// Menyalakan motor
```

```
void MotorOn() {
  isMotorRunning = true;
  if (dutyCycle == 0) dutyCycle = 200; // Beri nilai awal jika kecepatan 0
  applyMotor();
  Serial.print(">>> Motor ON, Kecepatan = "); Serial.println(dutyCycle);
  digitalWrite(ledPin, HIGH);
}

// Mematikan motor
void MotorOff() {
  isMotorRunning = false;
  digitalWrite(motor1Pin1, LOW);
  digitalWrite(motor1Pin2, LOW);
  ledcWrite(enable1Pin, 0); // Matikan sinyal PWM
  Serial.println(">>> Motor OFF");
  digitalWrite(ledPin, LOW);
}

// Menambah kecepatan sebesar DUTY_STEP
void naikkanKecepatan() {
  dutyCycle += DUTY_STEP;
  if (dutyCycle > DUTY_MAX) dutyCycle = DUTY_MAX; // Batasi
maksimal 255
  applyMotor();
  if (isMotorRunning) {
    Serial.print(">>> Kecepatan NAIK = "); Serial.println(dutyCycle);
  }
}

// Mengurangi kecepatan sebesar DUTY_STEP
void turunkanKecepatan() {
  dutyCycle -= DUTY_STEP;
  if (dutyCycle < DUTY_MIN) dutyCycle = DUTY_MIN; // Batasi minimal 0
  applyMotor();
  if (isMotorRunning) {
    Serial.print(">>> Kecepatan TURUN = "); Serial.println(dutyCycle);
  }
}

// Mengatur kecepatan ke nilai angka yang diketik manual
void setDuty(int val) {
  if (val < DUTY_MIN) val = DUTY_MIN;
  if (val > DUTY_MAX) val = DUTY_MAX;
  dutyCycle = val;
  applyMotor();
  if (isMotorRunning) {
    Serial.print(">>> Set Kecepatan = "); Serial.println(dutyCycle);
  }
}
```

```
// Setup Awal

void setup() {
  Serial.begin(115200);

  // Inisialisasi pin sebagai OUTPUT
  pinMode(ledPin, OUTPUT);
  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);

  // Konfigurasi pin PWM ESP32
  ledcAttach(enable1Pin, freq, resolution);

  MotorOff(); // Pastikan motor mati saat baru menyala

  Serial.println("\n--- Sistem Kontrol Motor Siap ---");
  Serial.println("Ketik: '1' (ON), '0' (OFF), '+' (Naik), '-' (Turun), 'PWM:150'
(Set Nilai)");
}

void loop() {
  // Membaca perintah dari pengguna melalui Serial Monitor
  while (Serial.available() > 0) {
    char c = Serial.read();

    // Jika menemukan karakter Enter (selesai mengetik)
    if (c == '\n' || c == '\r') {
      inputString.trim();

      // Mengecek apakah perintah berupa teks set nilai PWM (contoh:
PWM:150)
      if (inputString.startsWith("PWM:")) {
        int val = inputString.substring(4).toInt();
        setDuty(val);
      }
      // Mengecek apakah perintah berupa 1 karakter kontrol (1, 0, +, -)
      else if (inputString.length() > 0) {
        char cmd = inputString.charAt(0);
        if (cmd == '1')      MotorOn();
        else if (cmd == '0') MotorOff();
        else if (cmd == '+') naikkanKecepatan();
        else if (cmd == '-') turunkanKecepatan();
      }

      inputString = ""; // Kosongkan variabel untuk perintah berikutnya
    } else {
      inputString += c; // Rangkai karakter yang diketik menjadi kata
    }
```

```
  }
}
```

Penjelasan program

Secara arsitektur, program diawali dengan deklarasi pin kendali motor yang memanfaatkan topologi *H-Bridge* pada modul *driver* L298N. Pin 27 (IN1) dan Pin 26 (IN2) digunakan untuk menetapkan arah putaran motor yang dipatenkan bergerak maju melalui kombinasi logika HIGH dan LOW, sedangkan Pin 12 (ENA) difungsikan sebagai penerima sinyal *Pulse Width Modulation* (PWM).

Sinyal PWM dibangkitkan menggunakan fungsi ledcAttach() dengan menetapkan parameter frekuensi sebesar 30.000 Hz dan resolusi 8-bit. Pemilihan frekuensi tinggi (30 kHz) ini dilakukan secara spesifik agar gelombang beroperasi di atas ambang batas pendengaran manusia (20 kHz), sehingga secara efektif mengeliminasi suara dengungan (*whining noise*) pada kumparan motor saat beroperasi pada kecepatan rendah. Di sisi lain, resolusi 8-bit memetakan tingkat kecepatan motor ke dalam rentang rasio tugas (*duty cycle*) dengan nilai numerik dari 0 (berhenti) hingga 255 (kecepatan maksimal).

Perangkat keras dipisahkan ke dalam beberapa fungsi spesifik (MotorOn, MotorOff, naikkanKecepatan, turunkanKecepatan, dan setDuty). Seluruh fungsi modifikasi kecepatan tersebut bermuara pada pemanggilan fungsi sentral applyMotor(). Fungsi ini memuat logika pengaman (percabangan if(isMotorRunning)) yang memastikan sinyal tegangan hanya dikirim ke mikrokontroler apabila motor benar-benar dalam status dihidupkan (ON).

Selain itu, program mengimplementasikan mekanisme pengungkit torsi awal (*starting torque*). Ketika pengguna menghidupkan motor dari keadaan diam murni, mikrokontroler tidak memulainya dari nilai PWM yang terlalu rendah. Sistem secara otomatis memberikan lonjakan tegangan awal bernilai PWM 200 (if (dutyCycle == 0) dutyCycle = 200;) untuk melawan gaya gesek statis (*static friction*) sehingga motor langsung memiliki daya angkat untuk berputar.

Setiap modifikasi kecepatan juga dilindungi oleh algoritma *clamping* (pembatasan nilai). Algoritma ini menahan rentang nilai masukan agar tidak melewati batas minimum 0 (DUTY_MIN) dan batas maksimum 255 (DUTY_MAX). Pengaman ini sangat esensial untuk mencegah *integer overflow*, sebuah kondisi *bug* di mana motor bisa mati atau berperilaku tidak terprediksi akibat nilai memori yang melebihi kapasitas 8-bit.

Mekanisme interaksi antara pengguna dan perangkat keras dieksekusi melalui teknik penguraian teks (*string parsing*) yang beroperasi di dalam blok fungsi loop(). Mikrokontroler terus-menerus mengevaluasi antarmuka Serial Monitor (while

(Serial.available() > 0)) untuk membaca masukan ketikan karakter demi karakter, lalu merangkainya ke dalam variabel *buffer* inputString.

Begitu sistem mendeteksi karakter *Enter* (\n atau \r), program akan langsung mengeksekusi rangkaian teks tersebut. Jika pengguna memasukkan perintah khusus seperti "PWM:150", fungsi startsWith() akan mendeteksinya, lalu mengeliminasi empat karakter pertama menggunakan perintah substring(4) hingga menyisakan teks numerik murni ("150"). Teks tersebut kemudian dikonversi menjadi tipe data bilangan bulat melalui instruksi toInt() agar dapat dikalkulasi secara matematis oleh fungsi setDuty(). Untuk masukan karakter tunggal seperti '1', '0', '+', atau '-', program akan langsung memetakan karakter tersebut ke pemanggilan fungsi yang bersesuaian menggunakan percabangan if-else.

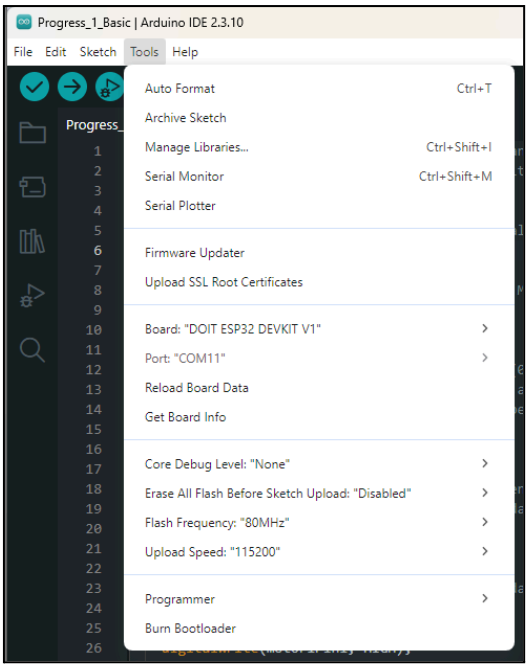
Kode program ini dirancang secara spesifik menggunakan arsitektur antarmuka pemrograman aplikasi (API) mutakhir dan tidak mendukung kompatibilitas mundur (*backward compatibility*) secara penuh. Kode ini secara eksklusif hanya dapat dikompilasi pada ESP32 Core versi 3.0.0 ke atas (seperti versi 3.3.12 yang digunakan pada tahapan *Setup*).

Pada sistem *legacy* (versi 2.x.x ke bawah), inisiasi PWM membutuhkan deklarasi kanal manual melalui ledcSetup() dan ledcAttachPin(). Namun, pada program ini, sintaks telah disederhanakan menggunakan fungsi abstraksi ledcAttach(pin, freq, resolution) dan ledcWrite(pin, dutyCycle) bawaan Core v3.x. Jika program ini dipaksa untuk diunggah menggunakan versi *board* lama, akan terjadi *Compilation Error*. Standardisasi lingkungan pengembangan ini membuktikan bahwa arsitektur perangkat lunak yang digunakan telah dioptimalkan sesuai dengan rilis teknologi mikrokontroler terkini.

1.6 Hasil Pengujian & Analisis

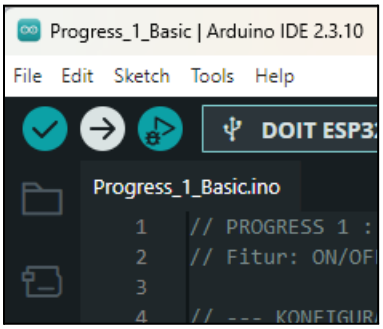
a. Proses Kompilasi dan Upload Program

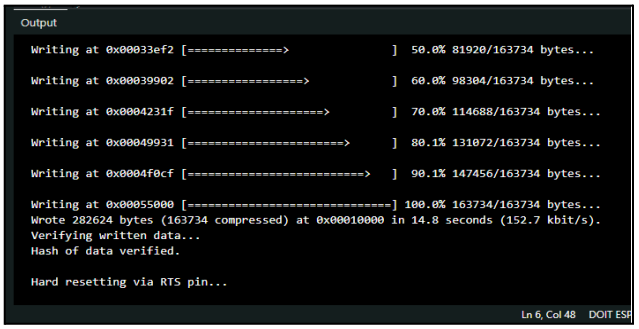
Sebelum melakukan kompilasi dan *upload* program, pastikan terlebih dahulu bahwa port komunikasi sudah terhubung dengan benar ke kit ESP32. Pastikan jenis *board* yang dipilih adalah DOIT ESP32 DEVKIT V1 dengan kecepatan *upload* (Upload Speed) sebesar 115200.



Gambar 1. Menu Tools Pemilihan Board
dan Port pada Arduino IDE

Jika pengaturan sudah benar dan siap, langkah selanjutnya adalah menekan tombol *Upload* untuk menanamkan kode ke dalam mikrokontroler.



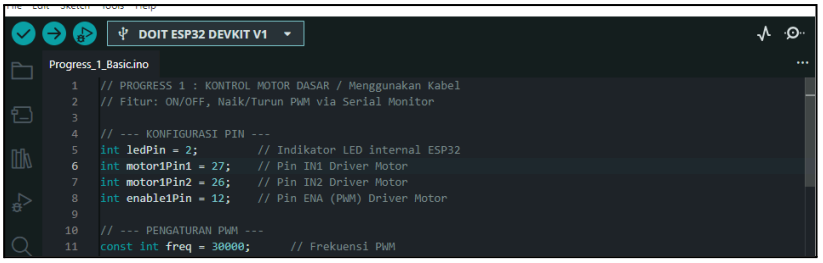


Gambar 2 dan 3. Proses Kompilasi
dan Upload Program

Tunggu sampai proses *upload* selesai. Jika kode berhasil diunggah tanpa galat (*error*), maka pada panel *Output* akan muncul log keterangan bahwa memori telah berhasil ditulis (tulisan "Wrote ... bytes" dan *Hard resetting via RTS pin*).

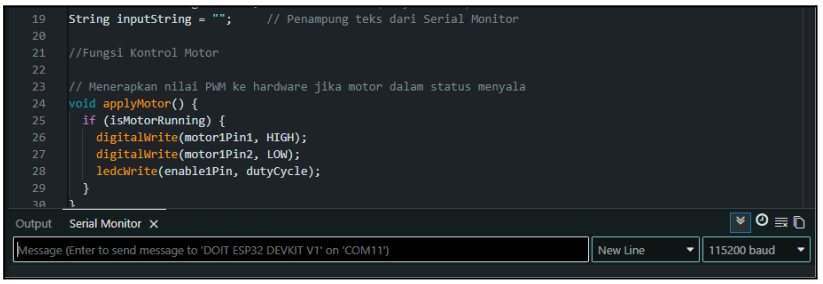
b. Pengujian Perintah via Serial Monitor

Setelah program berhasil ditanamkan, pengujian fungsionalitas dilakukan melalui antarmuka Serial Monitor. Langkah pertama adalah mengklik ikon menyerupai kaca pembesar di sudut kanan atas jendela Arduino IDE untuk membuka fitur tersebut. Fungsi utama dari Serial Monitor ini adalah untuk memberikan perintah teks (*input*) ke perangkat ESP32 sekaligus memantau status operasional mikrokontroler secara *real-time*.



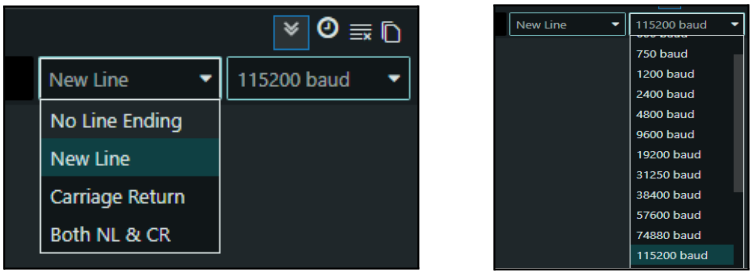
Gambar 4. Ikon Serial Monitor
pada Arduino IDE

Selanjutnya, jendela Serial Monitor akan terbuka pada bagian bawah layar (panel *Output*).



Gambar 5. Tampilan Jendela
Antarmuka Serial Monitor

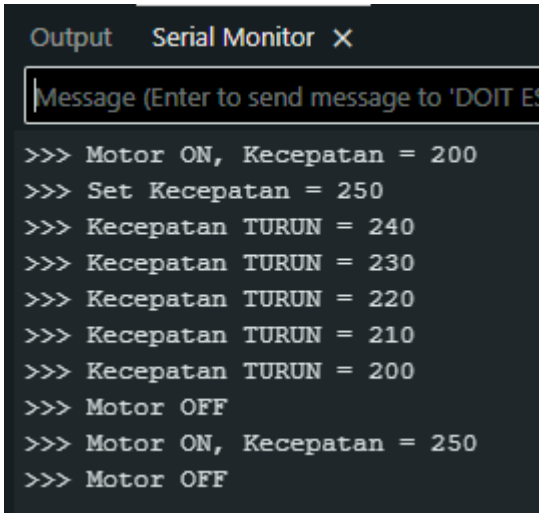
Sebelum melakukan uji coba pengiriman perintah ke motor, terdapat dua parameter krusial yang harus disesuaikan di pojok kanan jendela Serial Monitor. Pastikan pengaturan format baris (*Line Ending*) berada pada opsi New Line dan pengaturan kecepatan komunikasi (*Baudrate*) berada di angka 115200 baud agar selaras dengan kodingan Serial.begin(115200).



Gambar 6 dan 7. Pengaturan Opsi
Line Ending dan Baudrate

Jika konfigurasi sudah dipastikan sesuai, pengujian dapat langsung dilakukan dengan memasukkan karakter perintah instruksi yang telah diprogram. Berikut adalah skenario pengujian yang dilakukan: Pada tahap awal, perintah '1' dikirim untuk menyalakan motor, dan sistem secara otomatis memberikan nilai *kickstart* PWM awal sebesar 200. Selanjutnya, nilai PWM ditingkatkan secara manual ke angka 250 (menggunakan perintah PWM:250). Setelah itu, kecepatan diturunkan secara bertahap hingga menyentuh nilai 200 menggunakan perintah '-'. Motor kemudian dimatikan dengan perintah '0'. Saat motor dihidupkan ulang, mikrokontroler berhasil mempertahankan nilai memori PWM terakhir di angka 250.

Berikut adalah log interaksi hasil uji coba tersebut:



Gambar 8. Log Hasil Pengujian
Kontrol Motor via Serial Monitor

c. Analisis Keseluruhan

Berdasarkan hasil pengujian di atas, sistem ESP32 merespons perintah teks dari antarmuka Serial dengan sangat akurat dan presisi tanpa adanya *delay* (jeda) yang berarti. Kecepatan putaran motor berhasil diregulasi dengan halus melalui manipulasi nilai PWM sesuai *input* yang diberikan (seperti 1, 0, +, -, dan set nilai PWM manual).

Metode Serial murni ini memiliki keunggulan utama pada tingkat keandalannya yang mutlak karena tidak bergantung pada stabilitas koneksi jaringan nirkabel (Wi-Fi), sehingga sangat ideal untuk tahap kalibrasi awal. Namun, kelemahan utamanya adalah ketiadaan mobilitas operasional; perangkat keras harus senantiasa terhubung secara fisik ke PC/Laptop menggunakan kabel USB agar dapat menerima instruksi.